

Adding <=> to library

Contents

1. [Revision History](#)
2. [Introduction](#)
 - 2.1. [Motivation](#)
 - 2.2. [Non-member, hidden friend, or member?](#)
3. [Dealing with operator==\(\) and operator!=\(\)](#)
 - 3.1. [Most operator!=\(\)s are obsolete](#)
 - 3.2. [Reversed operator==s too](#)
 - 3.3. [Defaulting operator==\(\)](#)
4. [Adding <=> to std::basic_string](#)
 - 4.1. [Add a type alias](#)
 - 4.2. [Add a static member function](#)
 - 4.3. [Proposal](#)
5. [Adding <=> to std::vector](#)
 - 5.1. [Proposal](#)
6. [Adding <=> to std::optional](#)
 - 6.1. [Proposal](#)
7. [Adding <=> to std::unique_ptr](#)
 - 7.1. [Proposal](#)
8. [Acknowledgements](#)
9. [References](#)

§ 1. Revision History

A draft of this paper was presented to LEWG in Kona, and it was approved with the following details:

- comparison operators should be hidden friends
- `basic_string`'s comparison category should be driven by a [type alias](#), not a [static member function](#).
- tuple's `operator==` cannot be defaulted, because subobject order is currently unspecified. Defaulted `==` compares in subobject order, but tuple is specified to compare in template parameter order - so this would either force an ABI break on those implementations that don't order the subobjects from front to back or break the actual equality semantics. `pair` and `array` have no such problems with defaulting `operator==`.
- `unique_ptr` cannot have its relational operators removed due to the `less` specification - it ends up fitting in the `optional` category. `shared_ptr` is fine as proposed here.

§ 2. Introduction

This paper builds on all of David Stone's effort in [P0790R2](#) and exists just to fill in the missing holes. I believe it's very important to ship C++20 with `<=>` for all of these types - and this is my attempt to help get there. Of the types which P0790R2 does not provide a solution, they can be divided into four neat categories, each with a canonical representative. This paper will go through these four in turn:

- `std::basic_string`
- `std::vector`
- `std::optional`
- `std::unique_ptr`

Critically, no change proposed in this paper will change the semantics of any comparison. The answer that a particular comparison gave in C++17 will be the same as what that code will give in C++20. But the path taken to get to that answer may change.

§ 2.1. Motivation

But why should we bother? What's the motivation for doing all this work?

Major benefits of `operator<=>` are that it is easier to implement than the status quo comparisons implemented via `operator<`, that it can provide better performance than the status quo, and that it can be more correct than the status quo. But of course, none of that is true if we don't actually use it in our own library. There is the inevitable user frustration that this doesn't work:

```
struct Person {
    std::string last;
    std::string first;
    auto operator<=>(Person const&) const = default;
};
```

Here is a type ready-made for a three-way comparison (indeed, it already has one), but defaulting doesn't work? So much for ease of use!

A user might then, if [P1186R1](#) is adopted, try to do this:

```
struct Person {
    std::string last;
    std::string first;
    strong_ordering operator<=>(Person const&) const = default;
};
```

This will, potentially, walk each string twice - when we could have performed this comparison by walking each string only once (at most). So much for performance!

We don't even need to go into user types to see this issue. Consider:

```
void func(std::vector<std::string> const& names, std::vector<std::string> const& other_names)
{
    if (names < other_names) {
        // ...
    }
}
```

Even though `std::string` already has a three-way comparison function (by way of `compare()`), the invocation of `<` above won't use it - it'll instead invoke `<` up to twice on each string. That's twice as much work as needs to be done to answer this question. If we adopt `<=>` for all of these library types as we should, then the above comparison would go through `std::vector`'s `<=>` which would go through `std::string`'s `<=>` which ensures that we only compare each `string` at most one time. We're not just talking about making comparisons for user-defined types easier to write and more performant, we're also talking about making already existing comparisons of standard library types more performant.

And these problems stack, as the standard library types are inevitably used together. A `std::optional<std::vector<std::string>>` should provide an `operator<=>` with a comparison category of `strong_ordering`, which should be efficient, and it should really be there for C++20.

2.2. Non-member, hidden friend, or member?

This paper proposes adding a whole lot of `operator<=>`s into the standard library. There are three ways we could do this:

- non-member functions
- hidden friends
- member functions

Most (if not all?) of these operators are non-member functions today. This has the downside of polluting scope and increasing then number of candidates for each of the comparisons that overload resolution has to look through. It has the additional downside of allowing surprising conversions in certain cases simply because of ADL (e.g. [LWG3065](#)).

This leads many people to rightly suggest to make operators hidden friends instead, which maintains the advantage of non-member functions by allowing conversions in both arguments but reduces the number of candidates for unrelated types and doesn't have surprise conversions in both arguments. However, it does introduce some new possibilities for conversions that did not exist with the non-member function approach. Consider (on [godbolt](#)):

```
template <typename T>
struct C {
```

```

#ifdef FRIEND
    friend bool operator<(C, C);
#endif

#ifndef FRIEND
template <typename T>
bool operator<(C<T>, C<T>);
#endif

void ex(C<int> a, C<int> b) {
    a < b; // #1
    std::ref(a) < b; // #2
    a < std::ref(b); // #3
    std::ref(a) < std::ref(b); // #4
}

```

With the non-member operator template (i.e. without `FRIEND` defined), only `#1` is a valid expression. But with the hidden friend, *all four* are valid. These kind of additional conversions may be surprising - and it is certainly possible that they could lead to ambiguities.

But with `operator<=>` we can actually take a third approach: member functions. Because `<=>` considers reversed candidates as well, a member function is sufficient:

```

struct X {
    X(int);
    bool operator==(X const&) const;
    strong_ordering operator<=>(X const&) const;
};

// all of these are ok
X{42} < X{57};
X{42} < 57;
42 < X{57};
42 == X{57};
X{42} != 57;

```

Using a member function does not pollute the global scope and does not add to the work that overload resolution has to do needlessly. It does have unique behavior where conversions are concerned though. If we return to the earlier example with `C<T>`:

```

template <typename T>
struct C {
    std::strong_ordering operator<=>(C);
};

void ex(C<int> a, C<int> b) {
    a < b; // #1
    std::ref(a) < b; // #2
    a < std::ref(b); // #3
    std::ref(a) < std::ref(b); // #4
}

```

With this implementation, `#1` compiles straightforwardly. But `#2` and `#3` are *also* valid. `#3` evaluates as `a.operator<=>(std::ref(b)) < 0` while `#2` evaluates as `0 < b.operator<=>(std::ref(a))`. Only `#4` is ill-formed in this case. Recall that in the non-member case, only `#1` was valid and in the hidden friend case, all four were.

There, however, does exist a situation where moving from non-member comparison operator templates to a member spaceship function can change behavior. It's not completely free. This example courtesy of Tim Song:

Non-member <code><</code> template	Member <code><=></code>
<pre> template <typename CharT, typename Traits, typename Alloc> struct basic_string { /* ... */ }; // #1 </pre>	<pre> template <typename CharT, typename Traits, typename Alloc> struct basic_string { // #3 auto operator<=>(basic_string const&) const; </pre>

<pre> 6 template <typename CharT, typename Traits, typename Alloc> 7 bool operator<(basic_string<CharT, Traits, Alloc> const&, basic_string<CharT, Traits, Alloc> const&); 10 struct B { }; 11 // #2 12 template<typename CharT, typename Traits, typename Alloc> 13 bool operator<(basic_string<CharT, Traits, Alloc> const&, B const&); struct C { operator string() const; operator B() const; }; ""s < C(); </pre>	<pre> }; 10 struct B { }; 11 // #2 12 template<typename CharT, typename Traits, typename Alloc> 13 bool operator<(basic_string<CharT, Traits, Alloc> const&, B const&); struct C { operator string() const; operator B() const; }; ""s < C(); </pre>
--	---

Today, this goes through the global `operator<` template (#2). The `operator<` taking two `basic_string` s (#1) is not a candidate because `C` is not a `basic_string` . However, if we change `basic_string` to instead have a member spaceship operator... then this spaceship (#3) suddenly not only becomes a candidate (since `C` is convertible to `string`) but actually becomes the best viable candidate because the tie-breaker preferring a non-template to a template is higher than the tie-breaker preferring a non-rewritten candidate to a rewritten one. The same thing would happen in the case of a hidden (non-template) friend.

This seems like a fairly contrived scenario. Though like all fairly contrived scenarios, it assuredly exists in some C++ code base somewhere. The most conservative approach would be to stay put and keep the proposed `operator<=>` s as non-member operator templates. But there are very clear benefits of making them member functions, so I think member `operator<=>` is still the way to go.

§ 3. Dealing with `operator==( )` and `operator!=( )`

Before getting to how to handle `operator<=>( )` for all of these situations, let's first talk about the equality operators. There are several changes to how these operators behave as a result of [P1185R1](#) (R0 was approved by EWG in San Diego, R1 will likely be moved in Kona - there is still an open design question, but it is about a part of that proposal which is irrelevant to this paper).

§ 3.1. Most `operator!=( )` s are obsolete

The first part of P1185R1 changes the candidate set for inequality operators to include equality operators. In other words, for types in which `a != b` is defined to mean `!(a == b)` , we no longer need to define `operator!=` . The language will simply do the right thing for us. Just about every `operator!=( )` in the library does this. Indeed, we have blanket wording in [operators] which lets us avoid having to write the boilerplate every time. The only exceptions to this are:

- the types in the group represented by `std::optional` , which will be discussed [later](#)
- `std::valarray` , for which `==` doesn't really mean equality anyway

The other ~250 declarations of `operator!=( )` can just be removed entirely. The semantics for all callers will remain the same.

§ 3.2. Reversed `operator==` s too

Some library types provided mixed-type equality operators. For example, the class template `std::basic_string` provides the following equality and inequality operators today:

```

// #1
template <typename CharT, typename Traits, typename Alloc>
bool operator==(basic_string<CharT, Traits, Alloc> const&, basic_string<CharT, Traits, Alloc> const&);

// #2
template <typename CharT, typename Traits, typename Alloc>
bool operator!=(basic_string<CharT, Traits, Alloc> const&, basic_string<CharT, Traits, Alloc> const&);

// #3
template <typename CharT, typename Traits, typename Alloc>
bool operator==(basic_string<CharT, Traits, Alloc> const&, CharT const*);

// #4
template <typename CharT, typename Traits, typename Alloc>

```

```

bool operator==(CharT const*, basic_string<CharT, Traits, Alloc> const&);

// #5
template <typename CharT, typename Traits, typename Alloc>
bool operator!=(basic_string<CharT, Traits, Alloc> const&, CharT const*);

// #6
template <typename CharT, typename Traits, typename Alloc>
bool operator!=(CharT const*, basic_string<CharT, Traits, Alloc> const&);

```

The previous section suggests removing the inequality operators (#2 , #5 , and #6) and just relying on their equality counterparts to be used as rewritten candidates. But P1185R1, in addition to allowing inequality to be able to be rewritten as equality, also allows equality to be reversed. That is, a source expression `a == b` can find `b == a` as a candidate.

This means that #4 in the above isn't necessary either, since we can rely on `"hello" == "hello"s` to invoke #3. We really only need #1 and #3 in the above declarations. In other words, in order to be able to provide full equality and inequality between a `basic_string` and its corresponding `CharT const*`, we just need to write a single operator (#3) instead of today's four.

This paper proposes removing all of these duplicated `operator==` declarations as well. The full list is:

- `error_code` / `error_condition`
- `optional<T>` / `nullopt`
- `unique_ptr<T,D>` / `nullptr_t`
- `shared_ptr<T>` / `nullptr_t`
- `function<R(Args...)>` / `nullptr_t`
- `move_iterator` / `move_sentinel<S>`
- `counted_iterator` / `default_sentinel_t`
- `unreachable_sentinel_t` / `I`
- `istream_iterator` / `default_sentinel_t`
- `istreambuf_iterator` / `default_sentinel_t`
- `filter_view::iterator` / `filter_view::sentinel`
- `transform_view::iterator` / `transform_view::sentinel`
- `iota_view::iterator` / `iota_view::sentinel`
- `take_view::iterator` / `take_view::sentinel`
- `join_view::iterator` / `join_view::sentinel`
- `split_view::outer_iterator` / `default_sentinel_t`
- `split_view::inner_iterator` / `default_sentinel_t`
- `complex<T>` / `T`
- `valarray<T>` / `valarray<T>::value_type`
- `leap` / `sys_time<D>`
- `sub_match` / `basic_string`
- `sub_match` / `value_type const*`
- `sub_match` / `value_type const&`

Note that even though this paper is not removing `optional<T>`'s or `valarray<T>`'s `operator!=` s as a whole, it can still remove the reversed `operator==` s and the corresponding reversed `operator!=` s.

3.3. Defaulting `operator==( )`

Another important aspect of P1185 is that it redefines "strong structural equality" to be based on a defaulted `operator==( )` rather than a defaulted `operator<=>( )`, which is the requirement for a type being usable as a non-type template parameter - a new C++20 language feature introduced by way of [P0732R2](#)

There are many library types which currently define their `operator==( )` as member-wise equality. Respecifying all of these `operator==( )` s as being `default` ed instead will not change any semantics or ABI, but will simply directly allow these types to be usable as non-type template parameters. So this paper proposes to do that for:

- `pair`
- `tuple`
- `array`

... and any library type that meets this member-wise equality condition that is missing from the above list.

§ 4. Adding `<=>` to `std::basic_string`

This group is composed of templates which implement their comparisons via `Traits::compare()`: `std::basic_string`, `std::basic_string_view`, and `std::sub_match`. Because `Traits::compare()` is already a three-way comparison, this seems like a trivial case. Just write this and ship it right?

```
template <typename CharT, typename Traits, typename Alloc>
auto operator<=>(basic_string<CharT, Traits, Alloc> const& lhs,
                basic_string<CharT, Traits, Alloc> const& rhs)
{
    return lhs.compare(rhs) <=> 0;
}
```

Unfortunately, since `lhs.compare(rhs)` is an `int`, this would unconditionally yield a `std::strong_ordering`. The result of this will be correct as far as all the binary comparisons though, but comparing arbitrary `basic_string`'s is not necessarily a `strong_ordering`. Indeed, the [canonical example](#) of providing a custom `Traits` for `basic_string` is to implement a case-insensitive comparison - which should be a `weak_ordering`. So we cannot simply pick `strong_ordering`. On the other hand, `std::string`'s `operator<=>` should *certainly* yield `strong_ordering` so we cannot simply pick `weak_ordering` either. We'll have to do something slightly more complex.

We could simply bless all the standard-mandated specializations of `std::char_traits` and say that those are `strong_ordering`, and any other trait is a `weak_ordering`. But that seems overly restrictive on users who might want to provide a custom traits which provides an ordering which is either strong or partial.

Instead, I propose one of two choices: add a member type alias to the traits expressing the comparison category, or add a new static member function performing a three-way comparison and yielding a comparison category.

§ 4.1. Add a type alias

This is the least intrusive option. Simply add a member alias to the standard-mandated specializations of `char_traits`:

```
template <> struct char_traits<char> {
    using comparison_category = strong_ordering;
};
```

And replace the four relational operators with an `operator<=>` whose return type is based on that category:

```
// exposition-only
template <typename T>
struct traits_category {
    using type = weak_ordering;
};

template <typename T>
    requires requires { typename T::comparison_category; }
struct traits_category {
    using type = typename T::comparison_category;
};

template <typename T>
using traits_category_t = typename traits_category<T>::type;

// actual implementation
template <typename CharT, typename Traits, typename Alloc>
class basic_string {
    // ...
    traits_category_t<Traits> operator<=>(basic_string const& rhs) const
    {
        return compare(rhs) <=> 0;
    }
    // ...
};
```

This works because `int`'s comparison category is `strong_ordering`, which is implicitly convertible to any other comparison category.

This is a fairly small change. The downside is that any future custom traits will still have to write `compare()` functions that return an `int` instead of one of the standard comparison categories. To that end, we could:

§ 4.2. Add a static member function

Add a static member function returning a comparison category:

```
template <> struct char_traits<char> {
    static constexpr strong_ordering compare_3way(const char* s1, const char* s2, size_t count)
    {
        return compare(s1, s2, count) <=> 0;
    }
};
```

And then use that if it is present:

```
template <typename CharT, typename Traits, typename Alloc>
class basic_string {
    // ...
    auto operator<=>(basic_string const& rhs) const
    {
        auto impl = [](CharT const* lhs, CharT const* rhs, size_t sz) {
            if constexpr (requires { Traits::compare_3way(lhs, rhs, sz); }) {
                return Traits::compare_3way(lhs, rhs, sz);
            } else {
                return static_cast<weak_ordering>(
                    Traits::compare(lhs, rhs, sz) <=> 0);
            }
        };

        auto cmp = impl(data(), rhs.data(), min(size(), rhs.size()));
        return cmp != 0 ? cmp : size() <=> rhs.size();
    }
    // ...
};
```

This is somewhat more involved than the previous alternative, but not by much. Since we'd still need to provide a version of `compare` that returns an `int`, this may not actually be worth it - since while it's very easy to convert an `int` to a comparison category (simply `<=>` it against `0`), there's no actual easy way of going the other way without branching. But it *does* offer a way to make a `basic_string` with a partial ordering, if that is desired.

§ 4.3. Proposal

I weakly favor the member type alias approach. Remove `operator<`, `operator>`, `operator<=`, and `operator>=` for `basic_string`, `basic_string_view`, and `sub_match` and replace them with `operator<=>`s which invoke `x.compare(y) <=> 0` cast to the appropriate comparison category (with argument adjustments as appropriate). This `operator<=>` should be a member function.

§ 5. Adding `<=>` to `std::vector`

There are many types in the standard library that are, fundamentally, sequences of objects. Those sequences can be heterogeneous or homogeneous, fixed-length or variable-length, but they're all still sequences. The way we order these sequences, the way we write the comparison operators for them today, is lexicographically. Basically, this general algorithm:

```
bool operator<(Sequence const& x, Sequence const& y) {
    size_t const N = min(x.size(), y.size());
    if (x[0] < y[0]) return true;
    if (y[0] < x[0]) return false;

    if (x[1] < y[1]) return true;
    if (y[1] < x[1]) return false;

    // ...

    if (x[N-1] < y[N-1]) return true;
```

```

    if (y[N-1] < x[N-1]) return false;

    return x.size() < y.size();
}

```

For variable-sized sequences, this means we perform up to $2N$ invocations of an underlying `operator<`. For fixed-size sequences, we can optimize out the last two comparisons here so we only do up to $2N-1$ such invocations.

There are two important things to point out about comparing sequences.

- Every underlying type must be totally ordered. We are assuming that once `x[i] < y[i]` and `y[i] < x[i]` are both `false` that these elements are equivalent. For partial orders, this will give nonsense answers - which is why the standard library today simply states that comparing sequences with unordered elements are undefined behavior (e.g. `vector<float>` with any `NAN`).
- We are basically *already* doing three-way comparisons here. It's not enough to know the result of `operator<` - we need to know the full three-way ordering between every pair of elements.

Since we are doing what looks a lot like a three-way comparison anyway, we can write a direct translation of the above algorithm to `operator<==>` as follows:

<code>operator<()</code>	<code>operator<==>()</code>
<pre> bool operator<(Sequence const& x, Sequence const& y) { size_t const N = min(x.size(), y.size()); if (x[0] < y[0]) return true; if (y[0] < x[0]) return false; if (x[1] < y[1]) return true; if (y[1] < x[1]) return false; // ... if (x[N-1] < y[N-1]) return true; if (y[N-1] < x[N-1]) return false; return x.size() < y.size(); } </pre>	<pre> template <typename T> weak_ordering __cmp(T const& a, T const& b) { if (a < b) return weak_ordering::less; if (b < a) return weak_ordering::greater; return weak_ordering::equivalent; } weak_ordering operator<==>(Sequence const& x, Sequence const& y) { size_t const N = min(x.size(), y.size()); if (auto c = __cmp(x[0], y[0]); c != 0) return c; if (auto c = __cmp(x[1], y[1]); c != 0) return c; // ... if (auto c = __cmp(x[N-1], y[N-1]); c != 0) return c; return x.size() <==> y.size(); } </pre>

The implementation at right has the same semantics as the implementation at the left, it gives the same answer in all cases... because really we're performing exactly the same operations in the same order. Just slightly restructured. But once we restructure this way, we can do a lot better by simply using `<==>` where possible:

<code>operator<()</code>	<code>operator<==>()</code>
<pre> bool operator<(Sequence const& x, Sequence const& y) { size_t const N = min(x.size(), y.size()); if (x[0] < y[0]) return true; if (y[0] < x[0]) return false; if (x[1] < y[1]) return true; if (y[1] < x[1]) return false; // ... if (x[N-1] < y[N-1]) return true; if (y[N-1] < x[N-1]) return false; return x.size() < y.size(); } </pre>	<pre> template <ThreeWayComparable T> auto __cmp(T const& a, T const& b) { return a <==> b; } template <typename T> weak_ordering __cmp(T const& a, T const& b) { if (a < b) return weak_ordering::less; if (b < a) return weak_ordering::greater; return weak_ordering::equivalent; } ???? operator<==>(Sequence const& x, Sequence const& y) { size_t const N = min(x.size(), y.size()); if (auto c = __cmp(x[0], y[0]); c != 0) return c; if (auto c = __cmp(x[1], y[1]); c != 0) return c; // ... if (auto c = __cmp(x[N-1], y[N-1]); c != 0) return c; } </pre>


```

    return x.size() <=> y.size();
}

```

`ThreeWayComparable` is a concept proposed in [P1188R0](#).

Now, I will claim that the implementation at right is superior to the implementation at left, even in the cases where the underlying objects don't provide `<=>`. Consider a normal type that might exist in any given code-base today:

```

struct Legacy {
    bool operator<(Legacy const&) const;
};

```

Let's build from this. If I compare two `pair<Legacy, int>` s - either using the preexisting `operator<` (the left implementation) or the suggested `operator<=>` (the right implementation), I'm going to do the same operations either way: at most two invocations of `Legacy::operator<`. Unsurprising. But what if I compare two `vector<pair<Legacy, int>>` s? As I mentioned earlier, we perform up to `2N` invocations of the `operator<` for each object... which means in this case we're performing up to `4N` invocations of `Legacy::operator<` if we stick with today's `operator<` implementation. This is because we have to compare each `pair` in both directions. But if we use the `operator<=>` rewrite for both `pair` and `vector`, we only have to perform up to `2N` invocations of `Legacy::operator<`! This is because we're no longer throwing away work.

Of course, if the underlying objects *do* provide `<=>`, this can be even better. Comparing two `vector<pair<string, int>>` will transition from up to `4N` invocations of `basic_string::operator<` to up to `N` invocations of `basic_string::operator<=>`. That's... a lot.

In addition to the free performance win (fewer potential comparisons) and free complexity win (fewer functions to write and providing an `operator<=>` makes it easier for users that use templates as members to write their own `operator<=>` s), there's also a nice semantics win that we get from such a transition. I mentioned earlier how these sequence types do not work for partially ordered subtypes. But for types which provide an `operator<=>` which returns `std::partial_ordering` - we can easily do the right thing and provide sound comparisons that are themselves partially ordered. Less undefined behavior, which typically would manifest as just wrong or nonsense answers.

§ 5.1. Proposal

Remove `operator<`, `operator>`, `operator<=`, and `operator>=` for `array`, `deque`, `forward_list`, `list`, `map`, `move_iterator`, `multimap`, `multiset`, `pair`, `set`, `tuple`, `unordered_map`, `unordered_multimap`, `unordered_multiset`, `unordered_set`, and `vector` (including `vector<bool>`). For each of them, add a `operator<=>` with the same comparison semantics as today which compares each type `T` as if by the exposition-only `__cmp` function template above. These `operator<=>` s should be member functions.

§ 6. Adding `<=>` to `std::optional`

This group is composed of templates which forward *all* of their operations to underlying types. No operator is written in terms of any other operator. Even `!=` is not defined in terms of `==`. This group includes `std::optional`, `std::variant`, `std::queue`, `std::stack`, and `std::reverse_iterator`.

Unlike the types in the previous section - which were all sequences of some kind that assume a total order - the types in this group do not assume any semantics at all. An important consequence is that they provide the correct answer for types that have partial orders:

```

struct Q {
    float f;
    bool operator==(Q const& rhs) const { return f == rhs.f; }
    bool operator<(Q const& rhs) const { return f < rhs.f; }
    bool operator>(Q const& rhs) const { return f > rhs.f; }
};

// partial order, so no trichotomy
Q{1.f} == Q{NAN}; // false
Q{1.f} < Q{NAN};  // false
Q{1.f} > Q{NAN};  // false

// the types in the previous category assume trichotomy
vector<Q>{1.f} == vector<Q>{NAN}; // false
vector<Q>{1.f} < vector<Q>{NAN};  // false
vector<Q>{1.f} > vector<Q>{NAN};  // true?!

// but the types in this category do not

```

```
optional<Q>{1.f} == optional<Q>{NAN}; // false
optional<Q>{1.f} < optional<Q>{NAN}; // false
optional<Q>{1.f} > optional<Q>{NAN}; // false
```

This design for `optional` and `variant` is a result of [P0307R2](#). I do not know why `queue` and `stack` even have comparisons, or why `reverse_iterator` behaves this way.

But the important thing is that we cannot change the semantics of `optional` - the previous solution of synthesizing a `weak_ordering` from applying `operator<` in both direction is not okay here. That would change the answer for some comparisons, and this paper will not change semantics.

It turns out we cannot synthesize a `partial_ordering` either. The only sound way to do such a synthesis would be:

```
partial_ordering __cmp_partial(Q const& a, Q const& b)
{
    if (a == b) return partial_ordering::equivalent;
    if (a < b)  return partial_ordering::less;
    if (b < a)  return partial_ordering::greater;
    return partial_ordering::unordered;
}
```

But this requires additional operations on `Q`. Today, I can compare two `optional<Q>`'s with `<` with `Q` only having `<` ... and importantly I can compare two `optional<Q>`'s with `>` with `Q` only having `>`. There's no way to synthesize a `<=>` for `optional<Q>` in this scenario that has both the same semantics and the same operational requirements. That is a non-starter.

However, it's still really important to add `<=>` to these types for the same reason it's important to add `<=>` to the other types. It's just more difficult to do. Our only option is to adopt a new `operator<=>` that is constrained on the underlying types having `<=>` ensuring that `operator<=>` is *more constrained than* each of the two-way relational operators. Such an implementation, based on P1188R0, might look like:

```
template <typename T>
class optional {
    // ...
    template <ThreeWayComparableWith<T> U>
    compare_3way_type_t<T,U> operator<=>(optional<U> const& rhs) const;
    {
        if (has_value() && rhs.has_value()) {
            return **this <=> *rhs;
        } else {
            return has_value() <=> rhs.has_value();
        }
    }
    // ...
};
```

Making the spaceship operator more constrained than the other relational operators ensures that it gets precedence over them during normal comparisons.

§ 6.1. Proposal

Add a new `operator<=>` for each kind of comparison for `optional`, `variant`, `queue`, `stack`, and `reverse_iterator` such that `operator<=>` is constrained on the relevant template parameters satisfying `ThreeWayComparableWith` (or just `ThreeWayComparable`), ensuring that `operator<=>` is the best viable candidate for all relational comparisons in code. This `operator<=>` should be a member function.

Do not remove any of the preexisting comparison operators for these types, including even `operator!=()`.

The exception is the comparisons between `optional<T>` and `nullopt`. Those do not actually depend on the semantics of `T` in any way - so in this particular case, we can remove the relational operators and all the reversed candidates and replace them with member `operator<=>`. Instead of the 12 operator functions we have today, we only need two - both of whose implementations are trivial:

```
template <typename T>
class optional {
public:
    bool operator==(nullopt_t) const { return !has_value(); }
```

```
strong_ordering operator<=>(nullopt_t) const { return has_value() <=> false; }  
};
```

§ 7. Adding `<=>` to `std::unique_ptr`

For `std::unique_ptr` and `std::shared_ptr`, the comparison of `<` does not use raw `operator<` but rather goes through `std::less`. However, they do this in a slightly different way: Comparing a `unique_ptr<T1,D1>` to a `unique_ptr<T2,D2>` goes through `less<common_type_t<unique_ptr<T1,D1>::pointer, unique_ptr<T2,D2>::pointer>>` whereas comparing a `shared_ptr<T1>` to a `shared_ptr<T2>` goes through `less<>`.

Either way, P1188R0 proposes a three-way comparison object named `std::compare_3way` that satisfies the requirement that the ordering is a strict weak order, using the same wording that we have for `std::less` today. We can use this object to implement the comparisons doing something like:

```
template <typename T, typename D>  
class unique_ptr {  
    // ...  
    template <typename T2, typename D2>  
    auto operator<=>(unique_ptr<T2,D2> const& rhs) const  
    {  
        using CT = common_type_t<pointer, typename unique_ptr<T2,D2>::pointer>;  
        return compare_3way<CT>()(get(), rhs.get());  
    }  
    // ...  
};  
  
template <typename T>  
class shared_ptr {  
    // ...  
    template <typename U>  
    auto operator<=>(shared_ptr<U> const& rhs) const  
    {  
        return compare_3way()(get(), rhs.get());  
    }  
    // ...  
};
```

§ 7.1. Proposal

Remove `operator<`, `operator>`, `operator<=`, and `operator>=` for `unique_ptr` and `shared_ptr` and replace them with `operator<=>`s which go through the proposed `compare_3way` function object in the same way that the preexisting comparisons do today. These `operator<=>`s should be member functions.

§ 8. Acknowledgements

Thanks to David Stone for all the rest of the library work. Thanks to Agustín Bergé, Tim Song, Herb Sutter, and Jonathan Wakely for discussing issues around these cases.

§ 9. References

- [\[LWG3065\]](#) "LWG 2989 missed that all path's other operators should be hidden friends as well" by Billy O'Neal III, 2018-11-13
- [\[P0307R2\]](#) "Making Optional Greater Equal Again" by Tony Van Eerd, 2016-03-15
- [\[P0732R2\]](#) "Class Types in Non-Type Template Parameters" by Jeff Snyder, Louis Dionne, 2018-06-06
- [\[P0790R2\]](#) "Effect of operator<=> on the C++ Standard Library" by David Stone, 2019-01-21
- [\[P1185R1\]](#) "<=> != ==" by Barry Revzin, 2019-01-22
- [\[P1186R1\]](#) "When do you actually use <=>?" by Barry Revzin, 2019-01-22
- [\[P1188R0\]](#) "Library utilities for <=>" by Barry Revzin, 2019-01-22
- [\[gotw.29\]](#) "GotW #29: Strings" by Herb Sutter, 1998-01-03